R&D Architecture

SRE Managed Static Secrets

By Pedro Nunes 4 min 104 1

Context

Yet Another way to manage SRE static secrets.

For the full context of why this is needed and the motivations behind please check the original design in `https://outsystemsrd.atlassian.net/wiki/spaces/RDSD/pages/3787030777/Even+more+detailed+secret+management+for+Kubernetes+services#SRE-managed-static-secrets`

Why we need another way? Cause the original design have some security problems on rings segregation. Meaning, its difficult to ensure that a secret for a positive ring cannot be accessed from a negative ring.

Also, we can do the same in a much easy way, easy to implement and to maintain while keeping the same requirements.

Diagram

AWS SSM Parameter Store

Why AWS SSM Parameter Store and not AWS Secrets Manager?

AWS SSM Parameter Store Secure Strings offers exactly the same level of security and encryption of AWS ASM. The only difference lies on features. AWS SSM Parameter Store doesn't support:

- Automatic Secrets Rotation:
  - Anyway we cant rotate automatically this kind of secrets. Normally they are generated by a third party provider. We just need a safe place to store them.
- Built-in password generator
  - In most cases we cant auto generate this kind of secrets. They are generated by the third party provider.

AWS Parameter Store is free for *standard* parameters and 8x less expensive than ASM for *advanced* parameters.
AWS Parameter Store supports AWS Resource Access Manager (RAM) and ASM doesn't. To share secrets between multiple accounts we will use RAM.

So, why AWS SSM Parameter Store and not AWS Secrets Manager?

- Because is cheaper
- Parameters are sharable by RAM and we don't need to copy secrets back and forward from one account to another with Terraform and pipelines like proposed in the original design.
- In the original design, the number of ASM Secrets needed per static secret is 1 + (num of ring accounts). With Parameter store and RAM we only need one parameter.

To encrypt parameters we will use one KMS key per ring. The KMS key will only be accessible to accounts belonging to that ring using KMS policies. A parameter encrypted with a specific ring KMS key cant be decrypted on another ring account.

AWS Resource Access Manager

AWS RAM is a service from AWS that allow us to share resources to multiple accounts. The advantage is that we don't need to create cross account IAM Roles to access a resource in another account. The shared resource will appear in the destination account like it was created there.

The IAM permissions of the shared resource is controlled at the source account leaving the responsibility of who and what can be done in the shared resource in the hands of the administrators of the source account.

This simplifies a lot the management of resources that need to be available in all stamp accounts like global parameters/secrets.

We will have one RAM share per ring. A share is a group of resources shared with one or more destination accounts. To filter the ring accounts we will use the corresponding AWS Organization OU.

Detailed Instructions

Per Ring

Per ring we need to create a RAM share, KMS key, a KMS policy per ring

1. RAM Share:

```
aws ram create-resource-share \
    --name ring-minus3
    --no-allow-external-principals
    --principals arn:aws:organizations::082641424215:ou/o-aessakk
```

the other rings ask @Carlos Marques or anyone from the IAM team.

2. KMS Key

```
aws kms create-key
```

This will output the key id that we need for the next step

3. KMS Key Alias. Its always good to create a key alias so we can quickly identity the key.

```
aws kms create-alias \
    --alias-name alias/ring-minus3
    --target-key-id <id of key from the previous step>
```

4. KMS key policy.

Create a file named key_policy.json

```
{
    "Sid": "Allow use of the KMS key for a ring",
    "Effect": "Allow",
    "Principal": {
        "AWS": "*"
    },
    "Action": [
        "kms:Decrypt",
    ],
    "Resource": "*",
    "Condition": {
        "ForAnyValue:StringEquals": {
            "aws:PrincipalOrgPaths": ["o-a1b2c3d4e5/r-ab12/ou-ab
        }
    }
}
```

Here you need to replace the `PrincipalOrgPaths` with the ring specific. Same as you did for the RAM share.

Attach the policy to key created in step 2

```
aws kms put-key-policy \
    --policy-name default \
    --key-id <id from the key created in step 2> \
    --policy file://key_policy.json
```

We need to create the parameter/secret and then attach it to the RAM share of the ring.

1. Create the secret

```
aws ssm put-parameter \
    --name "/ring/-3/global/grafanacloud/apikey" \
    --type "SecureString" \
    --value "MySuperImportantSecretValue" \
    --key-id <id of the key created in step 2 of the ring>
    --tier "Advanced"
    --overwrite
```

This example creates a secret for Grafana.

2. Attach the secret to the RAM share created in step 1 of the ring

```
aws ram associate-resource-share \
    --resource-share-arn <arn of the resource share created in ri
    --resource-arns <arn of the parameter created in the previous
```

Important note: At this point, the accounts under the selected ring will have access. However, it's important to note that if the Dev teams' code assumes a User or Service Account (SA), the roles associated to that User or SA **must have** the a policy attached to it that grants access to SSM on SRE's account. Please check the For Consumers sections to share an example policy.

For Consumers

For secret consumers you just need to get the parameter the usual way. The only thing different is that you need to specify the full ARN of the parameter. Like:

```
aws ssm get-parameter \
    --name arn:aws:ssm:us-east-2:123456789012:parameter/MySharedP
```

In case your application is assuming an IAM User or Service Account, it must have the following policy in one of its roles:

```
 "Effect": "Allow",
          "Resource": [
            "arn:aws:ssm:us-east-1:761126061413:parameter/*"
```

Updated Sep 19, 2025 Edit

To be more specific and add access strictly to that parameter of a given ring, remove the wildcard and change the placholders: `arn:aws:ssm:us-east-1:761126061413:parameter/<ring>/<parameter_name>`

Related articles

Add a comment KB-HOW-TO-ARTICLE 1